

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

Освітній компонент
«АЛГОРИТМИ І СТРУКТУРИ ДАНИХ»

ЗВІТ
З ПРАКТИЧНОЇ РОБОТИ № 3

Виконала:
студентка групи КН-25-1
Марусіченко А. О.
Перевірив:
доцент кафедри КІЕ
Сидоренко В. М.

Кременчук 2026

Тема: Алгоритми сортування та їх складність. Порівняння алгоритмів сортування

Мета: опанувати основні алгоритми сортування та навчитись методам аналізу їх асимптотичної складності.

Порядок виконання роботи

Завдання 1

Бульбашкове сортування – це простий алгоритм, який порівнює сусідні елементи і переміщує найбільший елемент у кінець списку. Повторює цей процес для всіх елементів до тих пір, поки список не буде відсортований.

Алгоритм бульбашкового сортування:

$i = 0$

while $i < N - 1$:

$j = 0$

 while $j < N - 1 - i$:

 if $a[j] > a[j+1]$:

$a[j], a[j+1] = a[j+1], a[j]$

$j += 1$

$i += 1$

Оцінка складності: Найгірший випадок – $O(n^2)$ – виникає, коли масив відсортований у зворотному порядку. Найкращий випадок – $O(n)$ – виникає, коли масив уже відсортований.

Порівняння із сортуванням вставлянням: Сортування вставлянням працює швидше на практиці, оскільки виконує менше перестановок елементів.

Чому бульбашкове сортування менш ефективне за сортування злиттям? Бульбашкове сортування працює повільніше за сортування злиттям, тому що виконує велику кількість порівнянь і перестановок елементів.

Завдання 2. Оцінити асимптотичну складність алгоритму сортування зливанням, скориставшись основною теоремою рекурсії.

Рекурентне рівняння часу роботи алгоритму: $T(n) = 2T\left(\frac{n}{2}\right) + O(n)$

Де: $a = 2, b = 2, d = 1$. Обчислимо $\log_b a = \log_2 2 = 1$.

Оскільки $d = \log_b a$, то складність алгоритму:

$$T(n) = O(n^1 \log n) = O(n \log n)$$

Завдання 3

Швидке сортування – це ефективний алгоритм, який вибирає опорний елемент, розбиває список на дві підсписки, менший і більший за опорний елемент, і потім рекурсивно сортує кожен з них.

Алгоритм швидкого сортування:

```
def quick_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    else:  
        pivot = arr[len(arr) // 2]  
        left = [x for x in arr if x < pivot]  
        middle = [x for x in arr if x == pivot]  
        right = [x for x in arr if x > pivot]  
        return quick_sort(left) + middle + quick_sort(right)
```

Оцінка складності: Асимптотична складність швидкого сортування у середньому та найкращому випадках ідентична сортуванню зливанням і становить $T(n) = O(n \log n)$, що пояснюється використанням у обох алгоритмах стратегії «розділяй і володарюй». Проте в найгіршому випадку швидке сортування поступається злиттю, оскільки його складність може падати до $O(n^2)$ через нерівномірний поділ масиву.

Висновки опанувала основні алгоритми сортування та навчилася методам аналізу їх асимптотичної складності.

Контрольні питання

1. **Що таке асимптотична складність алгоритму сортування і чому вона важлива для порівняння алгоритмів?** Це математична оцінка швидкості алгоритму при збільшенні обсягу даних. Вона важлива, бо дозволяє

спрогнозувати поведінку коду на мільйонах елементів ще до його написання та об'єктивно порівняти різні рішення.

2. Які алгоритми сортування мають квадратичну складність у найгіршому випадку? Поясніть, чому це може бути проблемою для великих обсягів даних. Бульбашкове сортування, сортування вибором та сортування вставленням. Для великих даних це проблема, бо час виконання зростає занадто стрімко.

3. В чому полягає перевага сортування злиттям над сортуванням вставками для великих наборів даних? Має складність $O(n \log n)$ навіть для великих наборів даних.

4. Які алгоритми сортування використовуються для сортування списків у стандартних бібліотеках мов програмування, таких як Python, Java або C++? У Python використовується Timsort, у C++ — Introsort.

5. Яка різниця між алгоритмами сортування злиттям і швидким сортуванням? У яких випадках краще використовувати кожен з цих алгоритмів? Алгоритм злиття стабільний та потребує додаткової пам'яті, а швидке сортування зазвичай швидше на практиці.

6. Які фактори слід враховувати при виборі алгоритму сортування для конкретної задачі? Розмір і тип даних, необхідну швидкодію, обсяг доступної пам'яті та стабільність сортування.